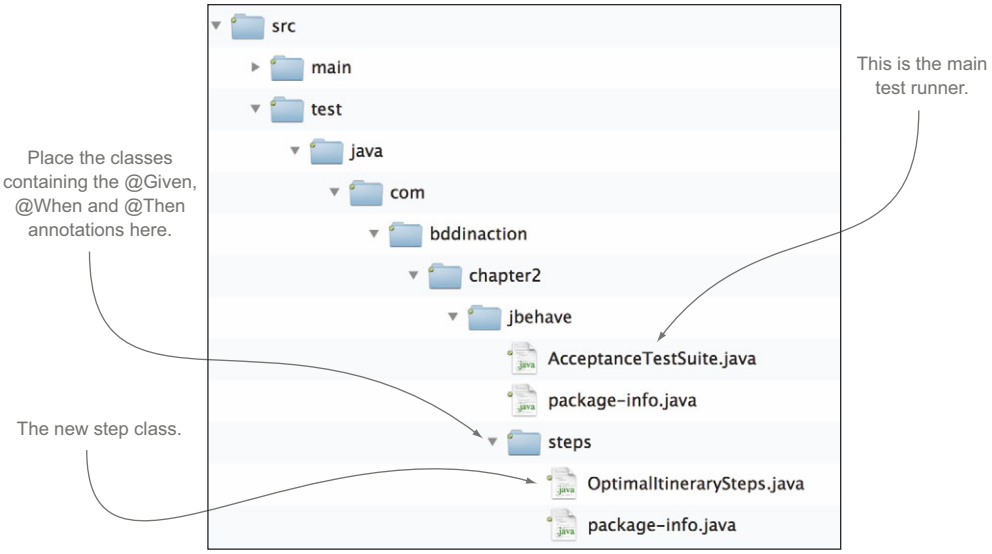


Figure 2.7 The package structure containing the new steps class

As it's written here, the `@Pending` annotation ❷ will ensure that this test will produce exactly the same results shown in figure 2.4. But as soon as possible, you'll want to flesh out the implementation of each method and turn this into a fully operational test.

These steps act as the starting point for your production code: they tell you what you need to build to get this feature out the door. BDD practitioners like to start with the outcome they need to obtain and work backwards, so let's see how that would work here.

Start with the `@Then` step, which expresses the outcome you expect. What you really want is for the service to give you a list of proposed train times that match the train times you expect. One way you could express this is as follows:

```
@Then("I should be told about the trains at: $expectedTrainTimes")
public void shouldBeInformedAbout(List<LocalTime> expectedTrainTimes) {
    assertThat(proposedTrainTimes).isEqualTo(expectedTrainTimes);
}
```

JBehave will match the first line of your scenario, extract the list of expected times (indicated by `$expectedTrainTimes` in the `@Then` annotation), and pass them to the method as a parameter. Note that this example also removes the `@Pending` annotation so that JBehave will know that it's supposed to execute this step. But this won't compile yet—you still need to decide where the proposed train will come from.

BDD helps you discover the technical design you need in order to deliver the business goals. Suppose you decide to implement this application using a number of different web services. For example, you may need an itinerary service that provides information about departure times to a website or a mobile app. This service doesn't exist yet, so you need to discover what this service should do. One of the most effective